

## COA Previous solve (19-20)

### 1(a) Learning outcomes from this course (4 marks)

Ans: এই course থেকে আমি শিখেছি –

1. Computer system ভিতরে কীভাবে কাজ করে — CPU, memory, I/O connection।
2. Instruction কীভাবে execute হয় এবং data কীভাবে move করে।
3. Pipeline, cache, memory management এর concept বুঝেছি।
4. ভবিষ্যতে system design, embedded system, এবং software optimization-এ এই knowledge কাজে লাগবে।

👉 In short: এই course আমাকে hardware-software relation এবং efficient system design বুঝতে সাহায্য করেছে।

---

### 1(b)(i) What is Computer Family? (1 mark)

একটা **Computer Family** মানে হলো — একই manufacturer-এর তৈরি এমন কিছু computer যাদের **architecture, instruction set একরকম**, কিন্তু **speed, size বা price আলাদা** হতে পারে।

👉 Example: Intel Pentium series, IBM System/360.

---

### 1(b)(ii) Find out the quotient and remainder by performing two's complement division operation where divisor = -4 and dividend = -7. (4 marks)

আমরা এখন **two's complement signed division** ওরকমভাবে করব, কিন্তু sign handle করে।

---

 **Given:**

- Dividend (Q) = -7
  - Divisor (M) = -4
- 

### Step 1: Sign আলাদা করি

Dividend negative  $\rightarrow$  sign bit = 1

Divisor negative  $\rightarrow$  sign bit = 1

✓ Sign of Quotient = Dividend  $\oplus$  Divisor =  $1 \oplus 1 = 0 \rightarrow$  Positive

✓ Sign of Remainder = Dividend = Negative

---

### Step 2: Unsigned মানে কাজ করবো

$$|-7| = 7$$

$$|-4| = 4$$

এখন unsigned divide করি:

$$7 \div 4 = 1 \text{ remainder } 3$$

---

### Step 3: Sign আবার যোগ করি

- Quotient positive থাকবে (কারণ  $- \div - = +$ )
  - Remainder dividend-এর মতো negative হবে
- 

### Step 4: Binary তে দেখাই (4-bit example)

| Value         | Decimal 4-bit Two's Complement |      |
|---------------|--------------------------------|------|
| Quotient (+1) | 1                              | 0001 |

| Value               | Decimal 4-bit Two's Complement    |
|---------------------|-----------------------------------|
| Remainder $(-3) -3$ | $\text{invert}(0011)=1100+1=1101$ |

---

✅ **Final Answer:**

- **Quotient = +1  $\rightarrow$  0001**
  - **Remainder = -3  $\rightarrow$  1101**
- 

🧠 **Easy Summary:**

Two's complement division-এ প্রথমে সবকিছু positive করে divide করা হয়, শেষে quotient এর sign বের হয়  $\oplus$  দিয়ে, আর remainder এর sign হয় dividend এর মতো।

---

**Final (Exam-Style Answer):**

Quotient = +1 ( $0001_2$ )

Remainder = -3 ( $1101_2$ )

---

**1(c)(i)  $(-A + B)$  in 6-bit Two's Complement (2 marks)**

ধরি,

$A = 12 \rightarrow 001100$

$B = 20 \rightarrow 010100$

$(-A + B) = ?$

**Step 1:**  $-A = \text{invert}(001100) + 1 = 110011 + 1 = 110100$

**Step 2:** Add B

110100

+010100

-----

1001000

6 bit রাখলে → 001000

✓ **Answer:**  $(-A + B) = 001000 \rightarrow 08 \text{ (Hex)}$

👉 সহজভাবে:  $-12 + 20 = +8 \rightarrow$  তাই Hex এ 08।

---

**1(c)(ii) Enlist the categories of computer's functions. What is the difference between arithmetic shift and logical shift? (1+2 marks)**

**Computer functions:**

1. Data processing (ডেটা প্রক্রিয়াজাত করা)
2. Data storage (ডেটা সংরক্ষণ করা)
3. Data movement (I/O কাজ)
4. Control operations (সমস্ত process নিয়ন্ত্রণ করা)

---

**Arithmetic shift vs Logical shift:**

| Type                    | Meaning                                          | Example        |
|-------------------------|--------------------------------------------------|----------------|
| <b>Arithmetic Shift</b> | Sign bit change হয় না (used for signed numbers) | 1010 →<br>1100 |
| <b>Logical Shift</b>    | Sign bit ignore হয়, নতুন জায়গায় 0 বসে         | 1010 →<br>0100 |

👉 **Easily:** Arithmetic shift sign রাখে, Logical shift শুধু bit move করে 0 বসায়।

---

অবশ্যই 😊 নিচে প্রশ্নসহ পুরো উত্তরগুলো **Bangla + English mix** করে একদম সহজভাবে দেওয়া হলো —

---

## 2(a) Technological features for switching computer generation (4 marks)

Computer generations mainly changed due to technology improvement in switching devices.

| Generation         | Technology Used            | Features                                                     |
|--------------------|----------------------------|--------------------------------------------------------------|
| 1st (1940–1956)    | Vacuum Tubes               | বড় size, বেশি heat, slow, huge electricity use              |
| 2nd (1956–1963)    | Transistors                | ছোট, faster, less heat, reliable                             |
| 3rd (1964–1971)    | Integrated Circuits (ICs)  | High speed, low cost, more compact                           |
| 4th (1971–1985)    | Microprocessors            | All CPU functions on single chip                             |
| 5th (1985–Present) | VLSI / AI-based processors | Parallel processing, machine learning, AI, quantum computing |

### 👉 Key technological development (recent consecutive generations):

- **4th → 5th:** From **microprocessor** to **AI & VLSI**, faster CPUs like Intel i7, multicore.
  - **5th → 6th (Current trend):** Quantum computing & AI chips (like Tensor cores, Neural Engines) evolving.
- 

## 2(b)(i) 2's Complement vs Sign Magnitude (2 marks)

| Point               | 2's Complement                                      | Sign Magnitude            |
|---------------------|-----------------------------------------------------|---------------------------|
| Representation      | Negative = invert bits + 1 MSB = sign bit (0=+,1=-) |                           |
| Arithmetic          | Simple to add/subtract                              | Complex (need sign check) |
| Zero Representation | Only one zero                                       | Two zeros (+0, -0)        |
| Example (4-bit)     | +5=0101, -5=1011                                    | +5=0101, -5=1101          |

→ **2's complement** is mostly used in computers because arithmetic is faster and simpler.

## 2(b)(ii) Variable Length Instruction (3 marks)

### Definition:

সব instruction একই size না হয়ে, কিছু ছোট কিছু বড় হয় — এগুলোকেই **variable length instruction** বলে।

### Example:

MOV R1, R2 → 2 bytes

MOV AX, [1234] → 3 bytes

Small instruction = faster fetching

Large instruction = handles more data/addressing

### Issues in Instruction Set Design:

1. **Complex decoding:** CPU কে আলাদা size চিনতে হয়।
2. **Memory alignment problem:** Unequal instruction boundaries.
3. **Hardware complexity:** Variable-length instruction handle করতে harder logic লাগে।

## 2(c) Condition Codes (2 marks)

**Definition:**

Condition codes are special **flags** (Zero, Carry, Negative, Overflow) that show the result status of operations.

**Advantages:**

- ✓ Speed up decisions (no need to recalculate conditions)
- ✓ Useful for branching (e.g., “if zero → jump”)

**Disadvantages:**

- ✗ Adds complexity in CPU design
- ✗ May cause confusion during parallel execution (flags shared)

---

**“Each operation needs operand” – Justify & Operand types (3 marks)****Justification:**

Every instruction needs **data to work on** — this data is called the **operand**. Without operand, CPU can't perform operation like ADD, SUB, LOAD.

**Example:**

ADD A, B → here A & B are operands.

---

**Categories of Operands:**

1. **Immediate Operand:** value directly in instruction  
→ e.g., MOV A, 5
  2. **Register Operand:** comes from CPU register  
→ e.g., ADD R1, R2
  3. **Memory Operand:** comes from memory location  
→ e.g., LOAD A, [2000]
  4. **I/O Operand:** comes from input-output device
-

### ✓ Final Short Summary (for exam):

- (a) Generations changed due to new switching devices (Vacuum → Transistor → IC → Microprocessor → AI).
- (b) 2's complement simpler than sign magnitude; Variable-length instructions = mixed size → complex decoding.
- (c) Condition codes = status flags; operands = data needed for operations.

---

### 3(a) Distinguish among Direct, Associative, and Set-Associative Mapping (2 marks)

| Mapping Type                                     | Explanation (Easy)                                                                                                                            | Example / Feature                                                                                                    |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>1 Direct Mapping</b>                          | প্রতিটি <b>main memory block</b> কেবলমাত্র <b>একটা নির্দিষ্ট cache line</b> -এ যেতে পারে।                                                     | Simple & fast but conflict বেশি হয়। Formula:<br>$\text{Cache line} = (\text{Block No}) \bmod (\text{No. of lines})$ |
| <b>2 Associative Mapping (Fully Associative)</b> | কোনো <b>memory block</b> cache-এর <b>যেকোনো line</b> -এ store হতে পারে।                                                                       | Very flexible, conflict কম, but hardware costly & slow searching.                                                    |
| <b>3 Set-Associative Mapping</b>                 | Cache কে <b>sets</b> এ ভাগ করা হয়; প্রতিটি <b>memory block</b> যায় নির্দিষ্ট set-এ, কিন্তু সেই set-এর মধ্যে <b>যেকোনো line</b> এ বসতে পারে। | Balance between direct & associative. Example:<br>2-way, 4-way set associative.                                      |

---

### 🧠 Easy summary:

- **Direct** → One fixed place only.



- **Associative** → Anywhere in cache.
- **Set-Associative** → In one set, but flexible inside that set.

✓ **Short answer (for 2 marks):**

Direct mapping fixes one cache line per block, associative mapping allows any line, and set-associative allows placement within a specific set — giving a balance of speed and flexibility.

**3(b)**

**i. For hexadecimal main memory addresses F000EE, FABCEF, show the following information in hexadecimal format. (Where cache size is 64 Kbytes.)**

**a. Tag, Line, and Word values for a direct mapped cache.**

**b. Tag, Set and Word values for two-way set associative cache. (5 marks)**

**Solve:**

**Given:**

- Main memory addresses: F000EE, FABCEF
- Cache size: **64 KBytes** =  $64 \times 1024 = 65536$  bytes
- **Direct mapped cache & 2-way set associative cache**

ধরা যাক **block size = 1 word = 1 byte** (যদি অন্য বলা না থাকে)

**(i) Direct Mapped Cache**

**Step 1: Determine number of cache lines**

- Cache size = 64 KB = 65536 bytes
- Block size = 1 byte → Number of lines =  $65536 / 1 = 65536$  lines

**Step 2: Address division**

- Main memory address: 24-bit (F000EE, FABCEF → 6 hex digits → 24 bits)
- For **direct mapped cache**:

$$\text{Address} = [\text{Tag} \mid \text{Line} \mid \text{Word}]$$

- $\text{Line} = \log_2(\#\text{lines}) = \log_2(65536) = 16 \text{ bits}$
- $\text{Word} = \log_2(\text{block size}) = \log_2(1) = 0 \text{ bits}$
- $\text{Tag} = \text{remaining bits} = 24 - 16 - 0 = 8 \text{ bits}$

✓ So:

- **Tag** = upper 8 bits
  - **Line** = next 16 bits
  - **Word** = 0 bits
- 

### Step 3: Address F000EE

- Binary: F0 00 EE  $\rightarrow$  Hex
- **Tag** = F0
- **Line** = 00EE
- **Word** = — (0 bits)

### Step 4: Address FABCEF

- Hex: FA BC EF
  - **Tag** = FA
  - **Line** = BCEF
  - **Word** = —
- 

## (ii) Two-Way Set Associative Cache

### Step 1: Determine number of sets

- Two-way  $\rightarrow$  2 lines per set
- Number of lines = 65536  $\rightarrow$  Number of sets =  $65536 / 2 = 32768$  sets
- **Set field** =  $\log_2(\#\text{sets}) = \log_2(32768) = 15 \text{ bits}$

- **Word** =  $\log_2(\text{block size}) = 0$  bits
- **Tag** = remaining =  $24 - 15 - 0 = 9$  bits

✓ So:

- **Tag** = upper 9 bits
- **Set** = next 15 bits
- **Word** = 0 bits

---

তাই, Address = [Tag: 9 bits | Set: 15 bits | Word: 0 bits]

---

### Step 2: Convert F000EE to binary

F000EE Hex → binary 24-bit:

F0 = 1111 0000

00 = 0000 0000

EE = 1110 1110

মিলিয়ে লেখা যাক 24-bit:

1111 0000 0000 0000 1110 1110

- **Tag:** প্রথম 9 bits = 111100000 → Hex?
  - Binary 111100000 → Decimal 480 → Hex = **1E0**
- **Set:** next 15 bits = 000000011101110 → Binary → Decimal 238 → Hex = **00EE**
- **Word:** 0 bits → —

✓ সুতরাং **F000EE** → Tag = **1E0**, Set = **00EE**, Word = —

---

### Step 3: Convert FABCEF to binary

FABCEF Hex → binary:

FA = 1111 1010

BC = 1011 1100

EF = 1110 1111

মিলিয়ে লেখা যাক 24-bit:

1111 1010 1011 1100 1110 1111

- **Tag:** প্রথম 9 bits = 111110101 → Decimal 501 → Hex = **1F5**
- **Set:** next 15 bits = 011110011101111 → Decimal 15583 → Hex = **3CDF**
- **Word:** 0 bits → —

✓ সুতরাং **FABCEF** → Tag = **1F5**, Set = **3CDF**, Word = —

---

**Final Answer (Correct)**

**Address Tag Set Word**

F00EE 1E0 00EE —

FABCEF 1F5 3CDF —

### b-ii) Cache Replacement Algorithms

- যখন cache এর set পূর্ণ হয়ে যায়, তখন কোন block কে replace করবে সেটা ঠিক করার জন্য কিছু algorithm থাকে। প্রধানগুলো হলো:
- ---
- **1. FIFO (First In First Out)**
- **English:** Replace the oldest block in the cache first.
- **Bangla:** যেটা সবচেয়ে আগে cache এ এসেছে, সেটা প্রথমে replace করা হয়।
- **Note:** সহজ, কিন্তু মাঝে মাঝে frequently used block ও replace হয়ে যেতে পারে।
- **Example:**  
Cache: [A, B, C]  
New block D আসলে, oldest A replace হবে।

- 
- - **2. LRU (Least Recently Used)**
  - **English:** Replace the block that has not been used for the longest time.
  - **Bangla:** যেটা সবচেয়ে দীর্ঘ সময় ধরে use হয়নি, সেটা replace করা হয়।
  - **Note:** FIFO থেকে better কারণ এটি temporal locality consider করে।
  - **Example:**  
Cache: [A, B, C]  
Recent use: A (recent), B, C (long time ago)  
New block D আসলে, C replace হবে।

- 
- - **3. LFU (Least Frequently Used)**
  - **English:** Replace the block used least number of times.
  - **Bangla:** যেটা সবচেয়ে কম বার use হয়েছে, সেটা replace করা হয়।
  - **Note:** ভাল যদি কিছু block অনেকবার use হয়।
  - **Example:**  
Cache: [A(5 times), B(2 times), C(7 times)]  
New block D আসলে, B replace হবে।

- 
- - **4. Random Replacement**
  - **English:** Replace a random block.
  - **Bangla:** যেকোনো random block replace করা হয়।
  - **Note:** সহজ, কিন্তু সবসময় optimal নয়।
  - **Example:**  
Cache: [A, B, C]  
New block D আসলে, randomly A বা B বা C replace হবে।

- 
- -  **সংক্ষেপে:**

- **FIFO:** সহজ, পুরানো block remove।
- **LRU:** বেশি recently used block রাখে।
- **LFU:** বেশি frequently used block রাখে।
- **Random:** randomly remove।

---

### 3(c)(i) Why is SRAM so called? Distinguish between SRAM and DRAM

#### Why SRAM is called SRAM?

- **English:** SRAM stands for **Static Random Access Memory**.
- **Bangla:** SRAM নামকরণ হয়েছে **Static Random Access Memory** থেকে।
- **Reason:** SRAM data কে **ধারাবাহিক power** থাকা পর্যন্ত **static** ভাবে রাখে, অর্থাৎ refresh করার দরকার হয় না।

---

#### Difference between SRAM and DRAM

| Feature           | SRAM                                            | DRAM                                           |
|-------------------|-------------------------------------------------|------------------------------------------------|
| Full Form         | Static RAM                                      | Dynamic RAM                                    |
| Data Storage      | Stores data <b>statically</b> (without refresh) | Stores data <b>dynamically</b> (needs refresh) |
| Speed             | Faster                                          | Slower                                         |
| Cost              | Expensive                                       | Cheaper                                        |
| Structure         | Uses <b>6 transistors per bit</b>               | Uses <b>1 transistor + 1 capacitor per bit</b> |
| Power Consumption | Less frequent switching                         | More due to refresh cycles                     |
| Usage             | Cache memory (CPU)                              | Main memory (RAM)                              |

#### Bangla Summary:

- **SRAM:** বেশি দ্রুত, বেশি ব্যয়বহুল, refresh লাগে না।

- **DRAM:** ধীর, সস্তা, refresh লাগে।

### 3(c)(ii) Explain the SRAM structure

- **English:** SRAM is made of **6 transistors per bit** forming a flip-flop. It stores **1 bit** of data in a stable state.
- **Bangla:** SRAM এ প্রতি bit **৬টি transistor** দিয়ে তৈরি হয়। এগুলো **flip-flop** হিসেবে কাজ করে এবং একটিমাত্র bit **stable ভাবে** রাখে।
- **Access:** Read/write সহজ এবং **fast**, কারণ refresh দরকার নেই।

#### Simple Diagram idea:

Q ----|>----+

Q' ---|>----+

(cross-coupled inverters form the flip-flop)

- Q = 1 বা 0 state ধরে রাখে।
- Access transistors connect bit line during read/write।

### 3(d) Distinguish between EPROM and EEPROM

| Feature      | EPROM                         | EEPROM                                  |
|--------------|-------------------------------|-----------------------------------------|
| Full Form    | Erasable Programmable ROM     | Electrically Erasable Programmable ROM  |
| Erase Method | <b>UV light</b> used to erase | <b>Electric signal</b> used to erase    |
| Rewrite      | Slower, needs UV erase        | Faster, can erase/write electrically    |
| Usage        | Firmware that rarely changes  | Firmware that may need frequent updates |
| Cost         | Cheaper                       | More expensive                          |
| Convenience  | Less convenient               | More convenient                         |

#### Bangla Summary:

- **EPROM:** UV light দিয়ে erase, ধীরে update।

- **EEPROM:** Electric signal দিয়ে erase, সহজে update।

---

💡 **Tip:** SRAM, DRAM, EPROM, EEPROM সবই memory এর প্রকার। মূল পার্থক্য হলো **speed, erase method, refresh requirement, structure**।

---

#### 4(a) What is RAID? Explain error detection and error correction mechanisms in semiconductor memory

##### RAID

- **English:** RAID = **Redundant Array of Independent/Inexpensive Disks**.
- **Bangla:** RAID হলো একাধিক হার্ড ডিস্ককে একসাথে যুক্ত করে ডেটা সংরক্ষণের একটি পদ্ধতি, যাতে data loss কমানো যায় এবং performance বাড়ে।
- **Purpose:**
  - Data redundancy (backup)
  - Faster read/write

##### Error Detection and Error Correction in Semiconductor Memory

###### 1. Error Detection

- Detects if memory data has **changed accidentally**.
- Methods:
  - **Parity bit** (extra bit 0 বা 1 add করা হয়)
  - **Checksum** (sum of data bits check)

###### 2. Error Correction

- Automatically **correct the wrong data**.
- Methods:
  - **Hamming code** (can correct 1-bit error)
  - **ECC (Error Correcting Code) memory**

**Bangla Summary:**



- **Error detection:** ভুল ধরার জন্য।
  - **Error correction:** ভুল ঠিক করার জন্য।
- 

#### 4(b) Technological features of storage devices

##### Device Feature (Bangla + English)

|              |                                                              |
|--------------|--------------------------------------------------------------|
| <b>Tape</b>  | Sequential access, cheap, large storage, slow read/write     |
| <b>CD</b>    | Optical storage, 700 MB, read mostly, portable               |
| <b>DVD</b>   | Optical storage, 4.7–8.5 GB, video/data, read/write possible |
| <b>HDD</b>   | Magnetic storage, fast access, mechanical spinning platters  |
| <b>SSD</b>   | Solid-state, fast, no moving parts, more expensive           |
| <b>Flash</b> | Non-volatile, small, portable, used in USB, memory cards     |

---

#### 4(c) Why I/O module is used & its functions

##### Why?

- **English:** To interface CPU with input/output devices.
- **Bangla:** CPU সরাসরি I/O device এর সাথে communicate করতে পারে না, তাই I/O module ব্যবহার করা হয়।

##### Functions of I/O module

1. Data transfer between CPU and I/O devices
2. Device control (start/stop operations)
3. Buffering (temporary storage)
4. Error detection in I/O

##### Bangla Summary:

- Data পাঠানো/গ্রহণ করা, device control, temporary storage, error check।
-

#### 4(d) Peripheral devices & classifications

##### Peripheral devices

- Devices outside the CPU & memory, like **keyboard, mouse, printer, scanner**.

##### Classification

1. **Input devices:** Keyboard, mouse, scanner
2. **Output devices:** Monitor, printer, speaker
3. **Input/Output devices:** Touchscreen, network card

##### Program-driven vs Interrupt-driven I/O

| Feature              | Program-driven | Interrupt-driven   |
|----------------------|----------------|--------------------|
| CPU waits for device | Yes            | No                 |
| Efficiency           | Low            | High               |
| Example              | Polling loop   | Keyboard interrupt |

---

#### 5(a)(i) Describe pipelining & branch handling

##### Pipelining

- **English:** CPU executes **multiple instructions simultaneously in stages**.
- **Bangla:** একাধিক instruction কে ধাপে ধাপে একসাথে process করা হয়, যেমন assembly line।

##### Branch Handling

- **Problem:** Branch instruction uncertainty can **stall pipeline**.
  - **Solution:**
    - Branch prediction
    - Delayed branching
    - Flush pipeline if mispredicted
-

### 5(a)(ii) Pipeline processor speed

- **Statement:** "k-stage pipeline processor is k times faster than non-pipeline."
  - **Reason:**
    - Non-pipeline: executes 1 instruction at a time.
    - Pipeline: executes **k instructions in k stages simultaneously** → increases throughput  $\approx k$  times.
- 

### 5(b)(i) Interrupts purpose & classification

#### Why interrupts?

- **English:** To let CPU handle important events immediately.
- **Bangla:** CPU কে তাত্ক্ষণিক কাজ করার সুযোগ দেয়, যেমন key press বা device ready।

#### Classification

1. **Hardware interrupt:** From devices (keyboard, mouse)
  2. **Software interrupt:** From programs (system call)
  3. **Maskable vs Non-maskable:** CPU can ignore maskable, must respond to non-maskable
- 

### 5(b)(ii) Handling multiple interrupts

1. **Priority scheme:** Assign priority to each interrupt
  2. **Interrupt nesting:** Higher priority interrupt can interrupt a lower priority one
  3. **Interrupt vector table:** CPU looks up which routine to run
- 

### 5(c)(i) Addressing mechanisms

- **Direct addressing**
- **Indirect addressing**
- **Immediate addressing**

- Register addressing
  - Indexed addressing
  - Base + displacement
- 

#### 5(c)(ii) Effective address $EA = (A) + (R)$

- **Formula:**  $EA = \text{Base} + \text{Displacement}$
- **A = Base value** (memory start address)
- **R = Register holding offset/displacement**

#### Example:

- Base (A) = 1000
- Register R = 20
- **$EA = 1000 + 20 = 1020$**

#### Bangla:

- Base address + register offset = final memory address (effective address)
- 

#### 💡 Tip for exams:

- সব memory, I/O, pipeline, interrupts-এর **tables বা diagrams** মনে রাখা সহজ।
  - Addressing এর জন্য ছোট example লিখে দেখাও।
- 

ঠিক আছে, আমি তোমাকে **সহজ বাংলায় + ইংরেজি** মিলিয়ে step by step বুঝিয়ে দিচ্ছি।

---

#### 6(a)(i) Number of Page Table Entries (PTE)

##### Formula:

$$\begin{aligned}
 &[ \\
 \text{Number of PTE} &= \frac{\text{Virtual Address Space}}{\text{Page Size}} = \frac{2^n}{2^p} \\
 &= 2^{n-p} \\
 &]
 \end{aligned}$$

**Virtual Address n Page Size P PTE =  $2^{(n-p)}$**

32-bit                      4 KB =  $2^{12}$      $2^{(32-12)} = 2^{20} = 1,048,576$

64-bit                      8 KB =  $2^{13}$      $2^{(64-13)} = 2^{51} \approx 2.25 \times 10^{15}$

**Bangla summary:**

- PTE = **কতগুলো page mapping লাগবে**, সেটা virtual address এবং page size থেকে হিসাব করা হয়।
- 

### **6(a)(ii) Page Fault Handling in Virtual Memory**

**Steps:**

1. CPU accesses a page → **page not in main memory** → page fault occurs.
2. OS **saves the current state** of the process (trap to OS).
3. OS **finds a free frame** in main memory.
4. If no free frame → **replace a page** using page replacement algorithm (FIFO, LRU, etc.).
5. **Load the required page** from disk to main memory.
6. **Update page table** with new frame number.
7. **Resume process** from where it was interrupted.

**Bangla:**

- Page fault হলে OS **disk থেকে missing page memory-তে load করে**, page table update করে process resume করে।
- 

### **6(b) Distinguish between CISC and RISC**

| <b>Feature</b>     | <b>CISC (Complex Instruction Set Computer)</b> | <b>RISC (Reduced Instruction Set Computer)</b> |
|--------------------|------------------------------------------------|------------------------------------------------|
| Instruction length | Variable                                       | Fixed                                          |

| Feature         | CISC (Complex Instruction Set Computer) | RISC (Reduced Instruction Set Computer) |
|-----------------|-----------------------------------------|-----------------------------------------|
| Instruction set | Complex, many instructions              | Simple, few instructions                |
| Execution time  | Multiple cycles                         | Usually 1 cycle per instruction         |
| Hardware        | Complex                                 | Simple                                  |
| Example         | Intel x86                               | ARM, MIPS                               |

#### Bangla Summary:

- **CISC:** বড়, জটিল instructions, slow, heavy hardware
- **RISC:** ছোট, simple instructions, fast, efficient

#### 6(c) Micro-operations for IAS architecture

**Instruction:** ADD AX, 3

#### Steps (simplified IAS style):

##### 1. Fetch:

- $MAR \leftarrow PC$  (Memory Address Register gets PC)
- $MDR \leftarrow M[MAR]$  (Memory Data Register gets instruction)
- $IR \leftarrow MDR$  (Instruction Register stores instruction)
- $PC \leftarrow PC + 1$

##### 2. Decode:

- Decode IR to identify opcode = ADD, operand = AX, 3

##### 3. Execute:

- $AC \leftarrow AC + 3$  (Accumulator adds 3)

#### Bangla Summary:

- Fetch → Decode → Execute → Update AC

## 6(d) CPU Performance Calculation

Given:

| Instruction Type | % occurrence | Cycles per Instruction |
|------------------|--------------|------------------------|
| ALU              | 40%          | 2                      |
| Load & Store     | 17%          | 3                      |
| Branch           | 40%          | 5                      |
| Others           | 3%           | 6                      |

- Total instructions = 350
- Clock rate = 1.2 GHz = 1/1.2e9 sec per cycle

### Step 1: Average CPI

$$CPI = \sum (\text{Fraction of instructions} \times \text{cycles per instruction})$$

$$CPI = 0.4 * 2 + 0.17 * 3 + 0.4 * 5 + 0.03 * 6 = 0.8 + 0.51 + 2 + 0.18 = 3.49$$

### Step 2: CPU Time

$$CPU\ Time = \frac{\text{Instruction Count} \times CPI}{\text{Clock Rate}}$$

$$CPU\ Time = \frac{350 \times 3.49}{1.2 \times 10^9} \approx \frac{1221.5}{1.2e9} \approx 1.018 \times 10^{-6} \text{ sec}$$

### Step 3: MIPS (Million Instructions Per Second)

$$MIPS = \frac{\text{Instruction Count}}{CPU\ Time \times 10^6} = \frac{350}{1.018e-6 \times 10^6} \approx 344MIPS$$

💡 Summary Table:

### Parameter Value

CPI            3.49

CPU Time    1.018  $\mu$ s

MIPS           344

### Bangla summary:

- CPI = **গড় cycles per instruction**
  - CPU Time = **total execution time**
  - MIPS = **million instructions per second**
-